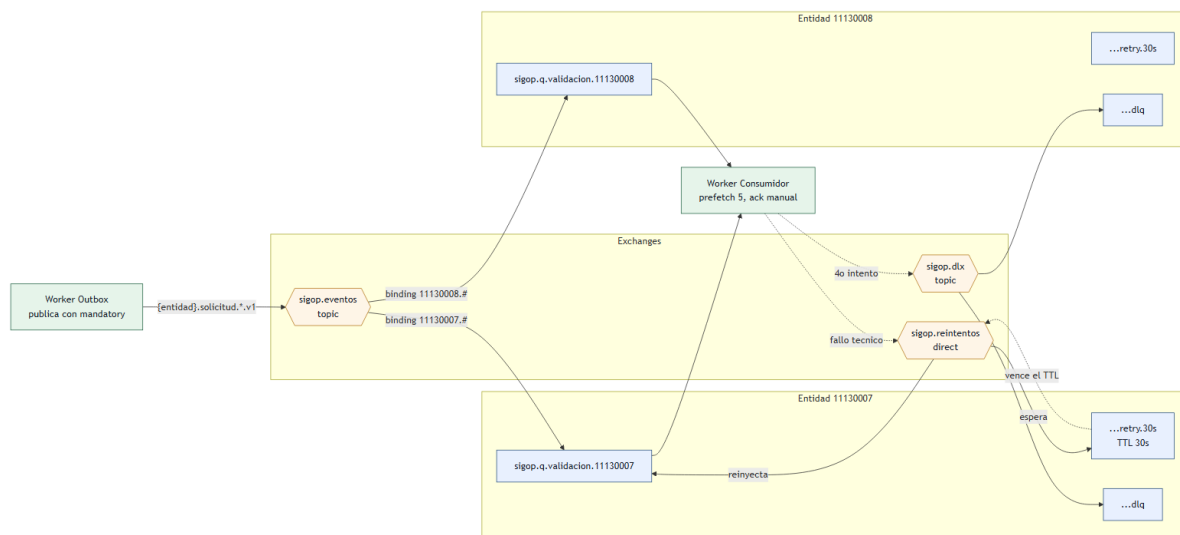


## Entregable 2. Diseño de mensajería

Topologia implementada y verificada en el piloto. Los nombres, valores y patrones de enrutamiento que aparecen aquí son los que declara la clase Topologia en Sigop.Infraestructura.

### 2.1 Topologia



Exchanges, colas por entidad, escalera de reintentos y cola de mensajes muertos.

La cola de espera **no tiene consumidor**: el mensaje entra, expira a los 30 segundos y RabbitMQ lo devuelve a la cola principal por su dead letter exchange. El retardo lo aplica el broker, no un hilo dormido en el consumidor.

### 2.2 Exchanges

| Exchange         | Tipo   | Para que                                                                                       |
|------------------|--------|------------------------------------------------------------------------------------------------|
| sigop.eventos    | topic  | Todos los eventos de dominio; el tipo topic permite que cada entidad se suscriba con un patron |
| sigop.reintentos | direct | Reinyeccion exacta a una cola concreta, sin patrones                                           |
| sigop.dlx        | topic  | Destino final de lo irrecuperable                                                              |

### 2.3 Colas

Tres por entidad. Con dos entidades cargadas son seis. Todas duraderas, no exclusivas y sin borrado automatico: sobreviven al reinicio del broker.

| Cola                                  | Argumentos                                        |
|---------------------------------------|---------------------------------------------------|
| sigop.q.validacion.{codigo}           | x-dead-letter-exchange: sigop.dlx                 |
| sigop.q.validacion.{codigo}.retry.30s | x-message-ttl: 30000, devuelve a sigop.reintentos |
| sigop.q.validacion.{codigo}.dlq       | Ninguno: es el final del camino                   |

## 2.4 Routing keys

```
{codigoEntidad}.{evento}
```

```
11130007.solicitud.registrada.v1  
11130007.solicitud.estado-cambiado.v1  
11130008.solicitud.estado-cambiado.v1
```

El binding de cada entidad es {codigoEntidad}.# , asi que una cola recibe todos los eventos de su entidad y ninguno de las demas.

**Por que una cola por entidad.** A favor: un pico de una entidad no retrasa a las demas, y la DLQ identifica al responsable sin inspeccionar mensajes. En contra: el numero de colas crece linealmente, con 300 entidades serian 900; y particionar la cola no particiona la dependencia, porque una caida de SICOIN afecta a todas por igual. Por encima de unas 50 entidades conviene agrupar por volumen, sin cambiar el patron de enrutamiento.

## 2.5 Contratos de evento y versionado

```
// solicitud.registrada.v1  
{  
  "solicitudId": "01a04945-b7c7-7b80-85f1-0a6e6bc1b07b",  
  "numero": "SOL-2026-000009",  
  "entidadId": 11130007,  
  "unidadEjecutoraId": 11130007001,  
  "tipoOperacion": "PagoProveedor",  
  "monto": 125000.00,  
  "moneda": "GTQ",  
  "creadoPor": "jperez"  
}
```

La version va **en la routing key**, no dentro del cuerpo. Publicar la v2 permite que convivan las dos: los consumidores viejos siguen atados a v1 y los nuevos se atan a ambas mientras dura la transicion.

| Cambio                                    | Sube la version? |
|-------------------------------------------|------------------|
| Agregar un campo opcional                 | No               |
| Quitar un campo o renombrarlo             | Si               |
| Cambiar el tipo de un campo               | Si               |
| Reutilizar un nombre con otro significado | Prohibido        |

El cuerpo no lleva datos sensibles en claro: el NIT y la cuenta bancaria viven cifrados en la base y el consumidor los descifra cuando los necesita.

## 2.6 Reintentos, TTL y DLQ

| Parametro              | Valor                                 |
|------------------------|---------------------------------------|
| Maximo de reintentos   | 3                                     |
| Retardo entre intentos | 30 segundos, aplicado por TTL de cola |
| Contador de intentos   | Cabecera x-intentos                   |
| Ultimo error           | Cabecera x-ultimo-error               |

Al agotar los intentos ocurren dos cosas, en este orden: la solicitud se marca como Fallida en la base, para que el usuario la vea y pueda reprocesarla, y el mensaje se envia a la DLQ con el numero de intentos y el ultimo error en cabeceras.

Verificado en ejecucion con el legado inalcanzable:

```
10s estado=REGISTRADA cola_reintento=1 dlq=0
70s estado=REGISTRADA cola_reintento=0 dlq=0 <- rebote del TTL
110s estado=FALLIDA cola_reintento=0 dlq=1 <- agotados los 3 intentos
```

## 2.7 Dos caminos de fallo distintos

| Tipo       | Ejemplo                                                 | Que ocurre                                                                   |
|------------|---------------------------------------------------------|------------------------------------------------------------------------------|
| De negocio | El banco rechaza; el legado responde sin disponibilidad | El dominio registra el desenlace, el mensaje se confirma y no hay reintentos |
| Tecnico    | Legado inalcanzable, base caída, error de transporte    | Excepcion: entra la escalera de reintentos y luego la DLQ                    |

Reintentar tres veces a un banco que ya respondió que no sería incorrecto: no es un fallo, es una respuesta. Solo se reintentará lo que puede salir distinto la próxima vez.

**Parking lot.** La DLQ retiene lo pendiente de análisis; el parking lot recibe lo ya analizado y no corregible. Política: revisión diaria de la DLQ, lo corregible vuelve a la cola principal por el exchange de reintentos, y lo demás pasa al parking lot con la causa documentada y 90 días de retención.

## 2.8 Acknowledgements y prefetch

autoAck en falso sin excepción: el mensaje se confirma **después** de que la transacción haya cerrado. Si el proceso muere entre medias, RabbitMQ reentrega y la idempotencia evita el doble efecto.

Prefetch de 5 por consumidor. Un valor alto acapara mensajes que otras instancias podrían atender; uno de 1 desperdicia el viaje de red. Es un número a ajustar con métricas reales.

## 2.9 Idempotencia

El identificador del mensaje lo genera el dominio al crear el evento, antes de que exista la fila:

```
public sealed record EventoDominio(string Tipo, string RoutingKey, object Contenido)
{
    public Guid EventoId { get; } = Guid.CreateVersion7();
}
```

Ese identificador es tres cosas a la vez: la llave de outbox\_mensajes, el MessageId que viaja en RabbitMQ y la llave primaria de mensajes\_procesados.

Lo correcto no es el identificador sino **donde** se escribe la marca: en el mismo SaveChanges que el efecto de negocio. La garantía es al menos una entrega mas idempotencia, que se comporta como exactamente una. Exactly-once de transporte no existe.

## 2.10 Orden

RabbitMQ garantiza el orden dentro de una cola con un solo consumidor, pero en cuanto hay varias instancias o un reintento, el orden se pierde. **El diseño no depende del orden.**

En su lugar, el consumidor decide por el estado actual de la solicitud, no por el tipo de evento recibido. Un mensaje que llega tarde o repetido encuentra un estado que ya no corresponde, lo registra y no hace nada. La maquina de estados rechaza cualquier transición ilegal, así que el desorden es inofensivo por construcción.

## 2.11 Correlación

Cada petición lleva la cabecera X-Correlation-Id; si no viene, la API genera una legible y fácil de dictar, con alfabeto de 32 caracteres sin cero, O, uno ni I:

K7M2-9PQR-4WXY

Ese identificador se guarda en la solicitud, en cada cambio de estado y en cada fila del outbox, y viaja en las propiedades del mensaje. La vista vw\_trazabilidad lo reúne todo en una sola consulta.

## 2.12 Estrategia de reproceso

Una solicitud fallida se reprocesa desde el detalle del portal, hasta 3 veces. El reproceso no reenvía el mensaje viejo: crea un evento nuevo, con su propio identificador, y devuelve la solicitud a EnValidación para que la saga la recorra otra vez desde un punto seguro.

El contador vive en la solicitud, no en el mensaje, y queda registrado en el historial con quien lo ordeno y por qué. Agotados los tres, la solicitud requiere intervención administrativa: el sistema deja de reintentar solo.

## 2.13 Publicación fiable

```
await canal.BasicPublishAsync(exchange, routingKey, mandatory: true, ...);
```

Con mandatory en verdadero, un mensaje que no encuentre cola destino se devuelve al publicador en lugar de descartarse en silencio; el despachador atiende BasicReturnAsync y lo registra. Sin esa bandera, un binding mal escrito produce mensajes que desaparecen sin rastro y solicitudes congeladas sin explicación: uno de los problemas que el enunciado describe en el sistema actual.